

(72 points possible)

Also needs the data file cereal.csv.

✓ Problem 1: Cereal (40 pts [1,2,2,4,3,6,5,3,6,8])

Download the cereal.csv file from the same place you downloaded this assignment; it's a dataset on 80 different cereal products from [Kaggle](#).

```
import pandas as pd
```

```
-----  
-  
ModuleNotFoundError                                Traceback (most recent call  
last)  
Cell In[1], line 1  
----> 1 import pandas as pd  
  
ModuleNotFoundError: No module named 'pandas'
```

```
# If using Google Colab, upload cereal.csv here
```

```
from google.colab import files  
uploaded = files.upload()
```

```
df = pd.read_csv('cereal.csv', index_col='name')  
df.head()
```

a) 1 pts) Find the mean, standard deviation, minimum value, 25th percentile value, 50th percentile value, 75th percentile value, and maximum value of the `calories`, `sodium`, and `rating` columns. (Printing these values for other columns too is fine.) You should refer back to your notes to find a DataFrame method that can do all of this in one call.

```
#TODO  
df.describe()  
df[['calories', 'sodium', 'rating']].describe()
```

b, 2 pts) Now write a line that prints just the `calories` column's statistics, rather than statistics for all the columns.

```
#TODO
df['calories'].describe()
```

c, 2 pts) Find a dataframe with all the cereals that have the median number of calories. (You can just "hard-code" the median that you saw in your results for parts (a) and (b). Note that `idxmax()` doesn't do what we want, as it just retrieves the name of one of these cereals.)

```
#TODO

median_cal = df['calories'].median()
df[df['calories'] == median_cal]
```

d, 4 pts) From all of the cereals with 110 calories, find the average sodium of these cereals.

```
#TODO
df[df['calories'] == 110]['sodium'].mean()
```

e, 3 pts) Find the correlation of the numeric variables. Are there any strong positive correlations between any of the data columns and the rating?

```
#TODO
df.corr(numeric_only=True)
```

There is a fairly positive correlation between the fiber and the rating. Surprisingly there is a negative correlation with the amount of sugar and the rating.

f, 6 pts) Use matplotlib to make a blue scatter plot of the fiber (x-axis) versus rating (y-axis). You may need to call `to_numpy()` on the relevant columns. For a frame of reference, plot the fiber values whose manufacturer (mfr) is Kellogg's ('K').

```
#TODO
import matplotlib.pyplot as plt

plt.scatter(df['fiber'], df['rating'], color='blue', label='All cereals')
plt.scatter(df[df['mfr']=='K']['fiber'], df[df['mfr']=='K']['rating'],
            color='orange', label="Kellogg's cereals")
plt.xlabel('Fiber')
plt.ylabel('Rating')
plt.title('Fiber vs Rating')
plt.legend()
plt.show()
```

g, 5 pts) Standardization is a procedure where a column of data is shifted and scaled to have a mean of 0 and a standard deviation of 1. This can be achieved by subtracting the mean of the column from each entry in the column and then dividing the whole column by the standard deviation. Create a new column called `st_rating` of the standardized ratings. Confirm this new column has a mean of 0 and a standard deviation of 1.

```
#TODO

# Standardize the rating column
df['st_rating'] = (df['rating'] - df['rating'].mean()) / df['rating'].std()

# Confirm the result
print(df['st_rating'].mean()) # Expect  $\approx 0$ 
print(df['st_rating'].std()) # Expect  $\approx 1$ 
```

```
print(df['st_rating'].mean()) # Expect 0 or a number on the order of  $e-16$ 
print(df['st_rating'].std()) # Expect a number very close to 1
```

h, 3 pts) Create a histogram of the standardized ratings using 25 bins. If we don't plot the histogram of the original (non-standardized) ratings, can we still determine whether the shape of the distribution has fundamentally changed?

```
#TODO
```

```
import matplotlib.pyplot as plt

plt.hist(df['st_rating'], bins=25, color='skyblue', edgecolor='black')
plt.xlabel('Standardized Rating')
plt.ylabel('Frequency')
plt.title('Histogram of Standardized Cereal Ratings')
plt.show()
```

Answer:

Standardization shifts and scales the data but does not change its overall shape. The histogram will look the same as the original (just re-centered around 0 and scaled).

i, 6 points) Write a function `name_popularity(df, word)` that iterates through the DataFrame `df` and finds the average `rating` of all cereals that contain `word` in their name. `word` can be any substring of the name ("in" is a substring of "find") rather than just space-delimited, but case should match. Assume the DataFrame's index contains the names, as with the DataFrame we've been using. Use `iterrows()` to perform this iteration.

For example, if the string "Corn" appears in the name of 4 cereals, with ratings 30, 60, 70, 40, then the return value is 50.

If the word isn't found in any name, return -1.

```
#TODO
def name_popularity(df, word):
    total = 0
    count = 0
    for idx, row in df.iterrows():
        if word in idx: # substring match in the cereal name
            total += row['rating']
            count += 1
    if count == 0:
        return -1
    return total / count
```

```
name_popularity(df, 'Corn') # Expect 40.48
```

```
name_popularity(df, 'Rice') # Expect 47.77
```

```
name_popularity(df, 'Silly') # Expect -1
```

j, 8 pts) We wish to create a column `Healthy` that indicates whether a cereal is healthy (using the value 1) or unhealthy (using the value of 0). Write a function `create_healthy_column(df, sodium_val, sugar_val, fiber_val)` that returns a DataFrame with this new column of 1's and 0's. The inputs to your function are the dataframe `df`, `sodium_val`, `sugar_val`, and `fiber_val`. A cereal is considered healthy when the amount of sodium is less than or equal to `sodium_val`, sugar is strictly less than `sugar_val`, and the fiber is greater than or equal to `fiber_val`. Print the names of the cereals that are healthy when `sodium_val=200`, `sugar_val=5`, and `fiber_val=2`.

```
#TODO
def create_healthy_column(df, sodium_val, sugar_val, fiber_val):
    df['Healthy'] = ((df['sodium'] <= sodium_val) &
                    (df['sugars'] < sugar_val) &
                    (df['fiber'] >= fiber_val)).astype(int)
    return df
```

```
#TODO
df = create_healthy_column(df, 200, 5, 2)
print(df[df['Healthy'] == 1].index)
```

✓ Problem 2: Regular Expressions (20 pts [10,6,4])

a, 10 pts) A recipe website uses the following URL format for its recipes ["https://awesomerecipes.com/recipes/abcdefgh-12/recipe-name"](https://awesomerecipes.com/recipes/abcdefgh-12/recipe-name). Users who post their recipes have unique ids (e.g., "abcdefgh-12") consisting of 8 letters followed by a hyphen and 2 numbers. The recipe name is either a single word or has a single hyphen separating the title, for example, "donuts" or "creme-donuts".

Write a function `extract_user_and_recipe(text)` that returns the unique ID and the recipe name from the URL. Ensure your function can extract all the words in the hyphen separated title.

```
#TODO
```

```
import re
```

```
def extract_user_and_recipe(text):  
    # Pattern: 8 lowercase letters, hyphen, 2 digits, then '/', then recipe name  
    pattern = r'(/[a-z]{8}-\d{2})/[a-z]+(?:-[a-z]+)*'  
    match = re.search(pattern, text)  
    if match:  
        user_id = match.group(1)  
        recipe_name = match.group(2)  
        return f"{user_id}, {recipe_name}"  
    else:  
        return None
```

```
# Expect 'abcxyzwv-34, donuts':  
text1 = 'https://awesomerecipes.com/recipes/abcxyzwv-34/donuts'  
# Expect 'abcxyzwv-34, donuts':  
text2 = 'https://awesomerecipes.com/recipes/abcxyzwv-34/creme-donuts'  
  
print(extract_user_and_recipe(text1))  
print(extract_user_and_recipe(text2))
```

b, 6 pts) A college's IT department has decided to update the unique IDs of their internal database of students and faculty/staff by changing to a new format. The old format consisted of 3 letter FAC for faculty, STA for staff, and STU for student, followed by 9 digits. The new format consists of a single letter, U for faculty/staff and u for students, followed by the original 9 digits. Write a function `convert_id(text)` which accepts as input a university ID and returns the re-formatted ID.

```
#TODO
```

```
def convert_id(text):  
    if re.match(r'FAC|STA', text):  
        # Faculty or staff → U + remaining digits  
        return 'U' + text[3:]  
    elif re.match(r'STU', text):  
        # Student → u + remaining digits  
        return 'u' + text[3:]  
    else:  
        return text
```

```
print(convert_id("FAC123456789")) # Expect 'U123456789'
print(convert_id("STA123456789")) # Expect 'U123456789'
print(convert_id("STU987654321")) # Expect 'u987654321'
```

c, 4 pts) An API (application programming interface) endpoint is a specific URL that allows clients (like web browsers, mobile apps, or other servers) to access specific resources or perform certain actions. API endpoints often have query parameters, for example `/data?XML` or `/users?id=123`. Write a function `extract_query(text)` which can extract queries that follow a `'?'` consisting of either 1 or more alphanumeric characters or 1 or more alphanumeric characters that are followed by an equal sign and a string of 1 alphanumeric characters. The return value should be a string.

```
#TODO
import re

def extract_query(text):
    # Pattern: matches '?' followed by one or more alphanumerics,
    # optionally followed by '=' and more alphanumerics
    match = re.search(r'\?([A-Za-z0-9]+(?:=[A-Za-z0-9]+)?)', text)
    if match:
        return match.group(1)
    else:
        return None
```

```
print(extract_query("/data?XML")) # Expect 'XML'
print(extract_query("/users?id=123")) # Expect 'id=123'
print(extract_query("/search?query=books")) # Output: 'query=books'
```

✓ Problem 3: Strings and DataFrames (10 pts)

Add a new column to the cereal DataFrame, *Bran perc*, that contains 1 for cereals that contain 100% Bran in the name, 0.5 if the name Bran without 100% is in the name, and 0 otherwise. You can assume that the 100% comes first in the name, but that Bran could appear anywhere else in the name.

Write a helper function `bran_perc(text)` first to help you; this should analyze the name and return the relevant number. Then iterate over `df.index` to build a list that you can assign to the new column.

```
#TODO
```

```
def bran_perc(text):  
    if "100%" in text and "Bran" in text:  
        return 1  
    elif "Bran" in text:  
        return 0.5  
    else:  
        return 0  
  
# Tests of your hypothetical helper function  
print(bran_perc('100% Natural Bran')) # Expect 1  
print(bran_perc("All-Bran")) # Expect 0.5  
print(bran_perc('Crispix')) #Expect 0
```

```
df.head() # Should see this working for Bran cereals
```

✓ AI Statement (2 points)

Please put an X in the box that best matches your AI use for this homework ([X] in the markup will become a checked box). There is no penalty for using AI, but we want to understand how you're using it.

- ☐ I did not use AI to complete this assignment.
- ☐ I didn't use AI to debug or generate code, but I did use it as a Python reference.
- ☒ I didn't use AI to generate code, but I did use it to debug some code.
- ☐ I did use AI to generate some code, but not all the code for any problem.
- ☐ I used AI to solve at least one problem completely.

When you are done, submit both your .ipynb (File->Download->Download .ipynb in Google Colab) and a PDF (File->Print->Save to PDF) to Blackboard where you found this assignment. (The backup PDF helps us give you points if there's a problem with your .ipynb.)

Start coding or [generate](#) with AI.

